University of Bari Aldo Moro
Department of Computer Science

One-For-All, TruthX and SLiMTruth: A study on the transferability between LLMs for hallucination detection and mitigation

Emanuele Fontana

March 4, 2026

Abstract

Large Language Models have demonstrated exceptional capabilities in numerous natural language processing tasks, yet they still suffer from the critical problem of "hallucinations," i.e., the generation of factually incorrect information presented with high confidence. Although analyzing the internal states of models (probing) has proven effective for detecting these anomalies, most current approaches require training classifiers specific to each individual architecture, limiting their scalability and general applicability.

This work explores the feasibility of a "universal prober" for hallucination detection, investigating whether and how detection capabilities learned on a "Trainer" model can be transferred to a different "Tester" model. We developed and compared various latent space alignment methodologies, ranging from hybrid approaches to non-linear projections.

The results demonstrate that transferring truthfulness-related features is highly effective, especially when using the *One-For-All* approach. It emerged that latent space alignment learned in simpler contexts is surprisingly robust and transferable even to more complex contexts, allowing high detection performance to be maintained. Conversely, alignment learned on complex and noisy data shows lower generalization capability toward simpler tasks.

This work highlights the potential of alignment techniques for creating cross-model safety systems, while also outlining the current limitations of internal representation transferability for advanced reasoning tasks.

Contents

1 Introduction **1**
- 1.1 Goals and Objectives . . . 1
- 1.2 Document Overview . . . 2

2 Background **5**
- 2.1 Transformer Overview . . . 5
  - 2.1.1 Transformer Architecture . . . 6
  - 2.1.2 Self-Attention Mechanism . . . 7
- 2.2 Large Language Models (LLM) . . . 8
- 2.3 Hallucinations in LLMs . . . 9
- 2.4 Probing . . . 9

3 Methodology **11**
- 3.1 Dataset . . . 11
  - 3.1.1 Datasets Used . . . 11
  - 3.1.2 Activation Dataset Creation . . . 13
- 3.2 LLMs Used . . . 14
  - 3.2.1 Qwen2.5-7B . . . 14
  - 3.2.2 Falcon3-7B-Base . . . 14
  - 3.2.3 Llama-3.1-8B-Instruct . . . 14
  - 3.2.4 Gemma-2-9B-IT . . . 14
- 3.3 Preliminary Studies . . . 15
  - 3.3.1 Hallucination Statistics . . . 15
  - 3.3.2 Studio delle singole componenti di layer . . . 16
- 3.4 Methodologies for Building the Universal Prober . . . 22
  - 3.4.1 FullLinear Approach . . . 23
  - 3.4.2 AdapterMLP Approach . . . 24
  - 3.4.3 Full Non-linear Approach . . . 26
  - 3.4.4 Reduced Non-linear Approach . . . 28
  - 3.4.5 One-For-All Approach (Frozen Head) . . . 31

4 Results **35**
4.1 Falcon3-7B-Base e Qwen2.5-7B su BeliefBankFacts 36
4.2 LLama-3.1-8B-Instruct e Gemma-2-9B-IT su BeliefBankFacts 37
4.3 LLama-3.1-8B-Instruct e Gemma-2-9B-IT su BeliefBankCalibration . . 38
4.4 LLama-3.1-8B-Instruct e Gemma-2-9B-IT su HaluEval 39

5 Discussion **41**
5.1 Comparison Between Methods 41
5.2 Dataset Analysis 41
5.3 Model Analysis 42
5.4 Layer Analysis 42

6 Cross-Domain Studies **43**

7 Conclusions **45**
7.1 Summary of Results 45
7.2 Future Work 45

Appendices **51**

List of Figures

2.1 Transformer Architecture 6
2.2 Self-Attention Mechanism Representation 7

3.1 Pipeline for extraction and saving of internal LLM activations 13
3.2 Performance of Qwen2.5-7B components on Belief Bank Facts 17
3.3 Performance of Falcon3-7B-Base components on Belief Bank Facts . . . 17
3.4 Performance of Llama-3.1-8B-Instruct components on Belief Bank Facts 18
3.5 Performance of Gemma-2-9B-IT components on Belief Bank Facts . . . 18
3.6 Performance of Llama-3.1-8B-Instruct components on Belief Bank Constraints 19
3.7 Performance of Gemma-2-9B-IT components on Belief Bank Constraints 19
3.8 Performance of Llama-3.1-8B-Instruct components on Halu Eval 20
3.9 Performance of Gemma-2-9B-IT components on Halu Eval 20
3.10 PCA of Attention activations of layer 18 of Falcon3-7B-Base 21
3.11 PCA of Attention activations of layer 14 of Qwen2.5-7B 21
3.12 FullLinear experimental pipeline 23
3.13 AlignmentNetwork architecture of the AdapterMLP approach 24
3.14 AdapterMLP approach experimental pipeline 25
3.15 MLP Prober architecture of the full non-linear approach 26
3.16 Full non-linear approach experimental pipeline 27
3.17 Autoencoder architecture of the reduced non-linear approach 28
3.19 AlignmentNetwork architecture of the reduced non-linear approach . . 29
3.18 MLP Prober architecture of the reduced non-linear approach 29
3.20 Reduced non-linear approach experimental pipeline 30
3.21 One-For-All encoder architecture 32
3.22 One-For-All Classification Head architecture 32
3.23 One-For-All experimental pipeline 33

List of Tables

3.1 Hallucination statistics for Belief Bank Facts 15
3.2 Hallucination statistics for Belief Bank Constraints 15
3.3 Hallucination statistics for Halu Eval 16
3.4 Layers chosen for Belief Bank Facts 21
3.5 Layers chosen for Belief Bank Constraints 22
3.6 Layers chosen for Halu Eval 22
3.7 AlignmentNetwork hyperparameter values apply to all combinations of Trainer, Tester and Layer Type. 26
3.8 AlignmentNetwork hyperparameters for the full non-linear approach. . 27
3.9 MLP Prober hyperparameters for the full non-linear approach. 28
3.10 AutoEncoder hyperparameters (Trainer and Tester). 30
3.11 AlignmentNetwork hyperparameters for the reduced approach. 31
3.12 MLP Prober hyperparameters (on latent space). 31
3.13 Encoder hyperparameters (Trainer and Tester Adapter). 33
3.14 Classification Head hyperparameters (Shared/Frozen). 33

4.1 Results for pair (Falcon3-7B-Base, Qwen2.5-7B) on BeliefBankFacts . . 36
4.2 Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on BeliefBankFacts 37
4.3 Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on BeliefBankCalibration 38
4.4 Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on HaluEval 39

6.1 Cross-Domain Results (One-For-All). Tr=Trainer, Te=Tester, BBC=BeliefBankConstraints, BBF=BeliefBankFacts, HE=HaluEval 44

Chapter 1

Introduction

1.1 Goals and Objectives

The public release of GPT-3 [3] by OpenAI marked a significant turning point in Natural Language Processing (NLP), paving the way for a new era of Large Language Models (LLMs) capable of maintaining conversations as fluently and naturally as a human. The release of the ChatBot was just the beginning: now LLMs are used in a wide range of applications and have become indispensable tools in private, academic, and industrial settings, with virtually unlimited applications.

However, one of the main obstacles to the widespread and safe adoption of these technologies in critical areas is the phenomenon of *hallucinations*. Despite syntactic coherence, LLMs occasionally tend to generate factually incorrect or fabricated information with a high degree of confidence. In contexts such as healthcare, legal support, or education, such hallucinations can lead to serious consequences (e.g., misdiagnoses, incorrect legal advice, or spread of misinformation). This behavior stems from the stochastic nature of LLMs: they cannot count or perform logical reasoning based on rules and facts; their ability to produce text depends solely on the statistics learned during the training phase on large text corpora. This is called *next token prediction*: given the context (i.e., the set of previous tokens), an LLM predicts the most probable next token based on learned distributions (we can think of a token as a word or part of it). A classic example of hallucination present in the earliest LLMs is:

> **User:** How many r's are there in the word strawberries?
> **LLM:** 2

Currently, techniques to mitigate this problem without making any intervention are often based on external fact verification (*retrieval-augmented generation*), specific *fine-tuning*, or *Chain of Thought* (CoT), but these techniques are not infallible, and it is therefore crucial to understand when a model is hallucinating and when it is not or to manipulate the generation process to reduce the probability of hallucinations.

Probing of internal activations (or *probing classifiers*) represents an innovative solution to the first problem. This methodology allows analyzing the internal state of

the model during generation, hypothesizing that information about the truthfulness of a statement is encoded in the latent space of neurons, regardless of the final textual output which is generated stochastically as it is influenced by multiple factors (temperature, top-k sampling, etc.). Several recent studies have demonstrated that it is possible to train simple linear classifiers to distinguish true statements from false ones by observing the activations of internal layers, and that generally activations related to hallucinations and those related to correct answers tend to be easily separable. [16]

Steering of the generation process is a promising approach to mitigate hallucinations. By manipulating the activations of specific neurons or layers during generation, it is possibile to guide the model's output towards desired behaviors [4]. Several studies have shown that it is possible to steer the generation process towards more truthful outputs to reduce hallucinations in LLMs.

Despite it has been shown that LLMs converge to a shared statistical model of reality, the so called *Platonic Representation Hypothesis* [10], which suggests that different LLMs may share a common internal representation of the world, including the phenomenon of hallucinations, most research has focused on creating probes and steering vectors specific to a single model or a specific architecture, leaving the problem of transferability relatively unexplored. This work aims to study whether there exists a universal representation of hallucinations in LLMs, regardless of the underlying architecture. In particular, the goal is to develop a general *prober* capable of detecting hallucinations in different models without the need for specific training for each of them and a general *steering* technique to mitigate hallucinations in different models (at most with a preliminary alignment between the latent spaces, as these differ between different models).

The main objectives of this project are:

- Analyze and compare the internal activations of different LLM families (e.g., Gemma, LLama) to identify common patterns associated with the hallucination phenomenon.
- Build a universal *prober* capable of transferring hallucination detection capability from one model to another without additional supervision on the target model.
- Develop a general *steering* technique to mitigate hallucinations in different models, with minimal or no need for model-specific adjustments.

1.2 Document Overview

The rest of the document is structured as follows:

- **Chapter 2: Background** - Theoretical background and overview of existing techniques for hallucination detection and mitigation in LLMs.
- **Chapter 3: Methodology** - Detailed description of the pipelines, the datasets, the models and the metodologies used to achieve the objectives of this work.

- **Chapter 4: Results** - Presentation of obtained results
- **Chapter 5: Discussion** - Discussion of obtained results
- **Chapter 6: Conclusions**: Final conclusions and proposal for future work

Chapter 2

Background

This chapter introduces the fundamental concepts necessary to understand the work carried out. The concept of attention, LLMs, the hallucination phenomenon, probing and steering will be presented.

2.1 Transformer Overview

The self-attention mechanism was first introduced in 2017 by Vaswani et al. in the paper Attention is All You Need [24] and is the foundation of the Transformer architecture, which has revolutionized the field of NLP and beyond, thanks to their ability to capture long-range relationships in sequential data more efficiently than previous models such as RNNs and their variants. The crucial difference between transformers and previous models lies in input processing: RNNs process data sequentially, maintaining a hidden state that captures previous information, while transformers use the self-attention mechanism to process the entire sequence simultaneously, allowing them to capture relationships between words regardless of their distance in the sequence. With very long inputs, RNNs tend to forget initial information while transformers do not have this problem.

2.1.1 Transformer Architecture

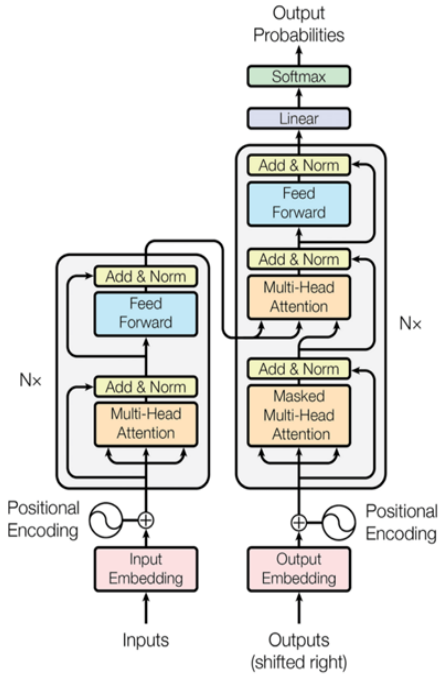

Figure 2.1: Transformer Architecture

A transformer is composed of two main blocks: *Encoder* and *Decoder*.

Encoder

The encoder consists of a series of identical layers placed in sequence, each of which contains:

- **Multi-Head Self-Attention**: this mechanism allows the model to focus on different parts of the input sequence simultaneously, capturing complex relationships between words.
- **Layer Normalization and Residual Connections**: to improve training stability and facilitate gradient flow, each sublayer is followed by normalization and a residual connection.
- **Feed-Forward Neural Network (FFNN)**: each encoder layer contains a fully connected feed-forward neural network that further processes the representations obtained from self-attention.
- Another Layer Normalization and Residual Connections layer.

The main role of the encoder is to transform the input sequence into an internal representation (embedding) that captures semantic relationships between words.

Decoder

The decoder also consists of a series of identical layers placed in sequence, each of which contains:

- **Masked Multi-Head Self-Attention**: similar to the mechanism in the encoder, but with a mask that prevents the model from "looking ahead" in the output sequence during training, ensuring that word generation occurs autoregressively.
- **Layer Normalization and Residual Connections**: similarly to the encoder, to improve training stability.
- **Multi-Head Attention on Encoder Output**: this mechanism allows the decoder to focus on relevant parts of the input representation generated by the encoder.
- Another Layer Normalization and Residual Connections layer.
- **Feed-Forward Neural Network (FFNN)**: as in the encoder, to further process representations.
- An additional Layer Normalization and Residual Connections layer.

2.1.2 Self-Attention Mechanism

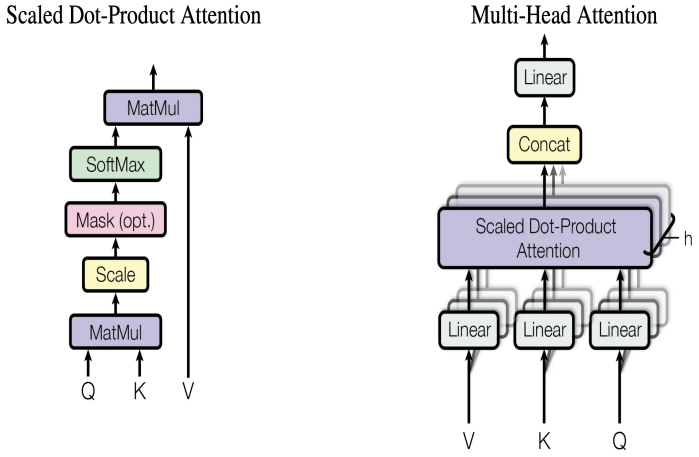

Figure 2.2: Self-Attention Mechanism Representation

The self-attention mechanism allows the model to weight the importance of different words in the input sequence when processing a particular word. Self-attention computation occurs through three distinct matrices: Query (Q), Key (K) and Value (V) [24].

The formula for computing attention is as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k is the dimension of the keys. The softmax normalizes attention scores, allowing the model to focus on the most relevant words. The multi-head attention mechanism extends this concept by performing multiple attention computations in parallel, allowing the model to capture different types of relationships between words.

2.2 Large Language Models (LLM)

LLMs represent the natural evolution of Transformer architectures applied at massive scale. An LLM is essentially a probabilistic model trained on enormous text corpora (on the order of trillions of tokens) with the fundamental goal of predicting the next token in a sequence, given the previous context. Formally, given a sequence of tokens $x = (x_1, x_2, ..., x_t)$, the model tries to estimate the conditional probability distribution $P(x_{t+1}|x_1, ..., x_t)$.

The evolution from previous NLP models to current LLMs (such as the GPT series, Llama, and Claude) is characterized by the *scaling law* phenomenon [12] according to which model performance improves predictably as the number of parameters, amount of training data, and computational power increase.

The typical lifecycle of an LLM is divided into distinct phases:

1. **Pre-training (unsupervised)**: This is the most computationally expensive phase. The model is trained in a self-supervised manner on vast datasets (web crawl, books, code) to learn language structure, logic, and a wide range of general knowledge. The objective is to minimize the *Cross-Entropy Loss* between the predicted distribution and the actual next token.

2. **Supervised Fine-Tuning (SFT)**: The pre-trained model (called Base) is further trained on a smaller, curated dataset of instructions and responses (prompt-response format). This allows the model to learn to follow user instructions and adopt a conversational format.

3. **Alignment**: To ensure the model is safe and aligned with human values, techniques such as *Reinforcement Learning from Human Feedback* (RLHF) or *Direct Preference Optimization* (DPO) are applied. In this phase, the model is penalized if it generates toxic content, biases, or unhelpful responses.

An emergent characteristic of LLMs is the capability of *In-Context Learning*, that is, the ability to learn new tasks simply by observing some examples in the prompt, without updating model weights.

2.3 Hallucinations in LLMs

Despite their extraordinary generative capabilities, LLMs suffer from a critical limitation known as hallucination. In the context of artificial intelligence, a hallucination occurs when a model generates output that is syntactically correct and fluent, but factually incorrect, logically inconsistent, or unfaithful to the provided input source. Huang et al. [9] define different types of hallucinations:

- **Factual Hallucinations**: The model produces information that is simply false or inaccurate with respect to reality.
- **Logical Hallucinations**: These occur when the model generates responses that are logically inconsistent or contradictory.
- **Context Inconsistencies**: These occur when the model contradicts the context provided in the input.

The causes of hallucinations are multiple and include: lossy compression of knowledge during training, noisy or contradictory training data, and the model's tendency to prioritize text fluency over factual accuracy, defined as *sycophancy*, i.e., the tendency to please the user by confirming their biases [18]. Detecting and mitigating hallucinations is currently one of the most open challenges in LLM research.

2.4 Probing

Probing (or *Probing Classifiers*) is an analysis technique used in the field of *Explainable AI* (XAI) to interpret the internal representations of deep neural networks. The fundamental idea is that, although LLMs are often considered black boxes, their internal activations (the values of hidden states and outputs of various modules at each layer) contain rich and structured information about the processed input. In particular, Marks and Tegmark [16] demonstrated that in LLMs the concepts of truth and falsehood emerge through linear structures in the space of the model's internal representations. It is therefore possible to use probers to probe these representations and verify whether an LLM is hallucinating or not. The linear probing strategy (described here) is certainly the most common (an example is SAPLMA [2]) :

1. **Activation extraction**: Input is provided to the model (frozen, i.e., with frozen weights) and the activation vectors h_l generated at a specific layer l are recorded.
2. **Prober training**: A simple supervised classifier (often a Linear Regression or a simple Multi-Layer Perceptron) is trained that takes as input the activations h_l and tries to predict a specific property y of the input (for example: part of speech, sentence length, or, in the case of this work, the truthfulness of the statement).

3. **Evaluation**: The classifier's performance indicates how much information related to property y is encoded linearly (or non-linearly) in that particular layer of the model.

but variants also exist.

Fadeeva et al. propose Claim-Conditioned Probability (CCP) [5] whose central idea is to compare the probability of the given statement with that of counterfactual variants generated by masking important tokens and regenerating conditionally. A low value suggests that the model itself considers the generated statement fragile.

Hewitt et al. instead propose *Structural Probing* [7] which aims to verify whether the internal representations of a language model contain information about the syntactic structure of sentences, such as dependency trees.

Manakul et al. propose *SelfCheckGPT* [15] which does not use activations but rather generates different responses to the same question and compares them: if they are consistent with each other, the model is not hallucinating, while if they are different, it means the model has hallucinated

Chapter 3

Methodology

This chapter explores the methodology adopted for the project development. For building the universal prober, the datasets Belief Bank Facts [13], Belief Bank Constraints [13] and Halu Eval [14] were used. Five different methodological approaches were considered to align the latent space of a tester model to that of a trainer model, where trainer model refers to the model whose activations are used to train the prober model, and tester model refers to the model whose activations are used to test the prober's transferability (after a possible alignment phase). The first phase of the work involved dataset preparation and extraction of internal activations from the LLMs across all layers and each type of layer component (attention, MLP, hidden states). Subsequently, preliminary studies were conducted to evaluate the ability of each individual layer component to discriminate between true statements and hallucinations, in order to identify the most relevant layers and components for building the universal prober. Some statistics related to hallucinations of the two models were then calculated, and all activations were preprocessed to be used in the various methodological approaches. In an initial phase, a completely linear approach was used, then more complex methods were explored including non-linear adaptations via AdapterMLP, Encoder and AutoEncoder. Each methodology was evaluated in terms of performance and transfer capability between different models.

3.1 Dataset

3.1.1 Datasets Used

Belief Bank Facts

This dataset was chosen as it can be used to detect *Factual Hallucinations*. The original BeliefBank facts cover 85 distinct entities belonging to the natural domain (animals and plants). Each instance in the dataset is a simple declarative sentence, such as *An eagle is a bird* (True) or *An eagle is a mammal* (False). The truth labels (called *silver labels*) were generated by the authors.

The final dataset is composed of:

- **Affirmative facts:** The original assertions extracted from BeliefBank (e.g., $fact$).
- **Negated facts:** For each present fact, its negated counterpart was synthetically generated (e.g., $\neg fact$), inverting the original truth label (from True to False and vice versa), in order to balance the classes and test model robustness.

with a total of 27,416 statements, of which 13,708 labeled as true (yes) and 13,708 as false (no), ensuring perfect balance between classes.

Belief Bank Constraints

This dataset was chosen as it can be used to detect *Logical Inconsistencies*. The original BeliefBank constraints were manually created by the authors and represent logical relationships between statements. The dataset is composed of:

- **Positive implications:** Representation of logical implications between statements (if A then B).
- **Mutual exclusions:** Representation of mutual exclusion constraints between statements (not both A and B).

For each element, the negated version was also inserted to balance the classes. In total there are 25,756 statements to verify.

HaluEval

This dataset was chosen to detect hallucinations in complex conversational contexts. Unlike simple facts, here each instance includes a context composed of dialogue history and supporting external knowledge.

The dataset is composed of:

- **Correct Responses:** Factual responses consistent with the knowledge provided in the context (Right Response).
- **Hallucinated Responses:** Synthetically generated responses that seem plausible but contain unverified or contradictory information with respect to the provided knowledge (Hallucinated Response).

The final dataset is composed of 10,000 examples.

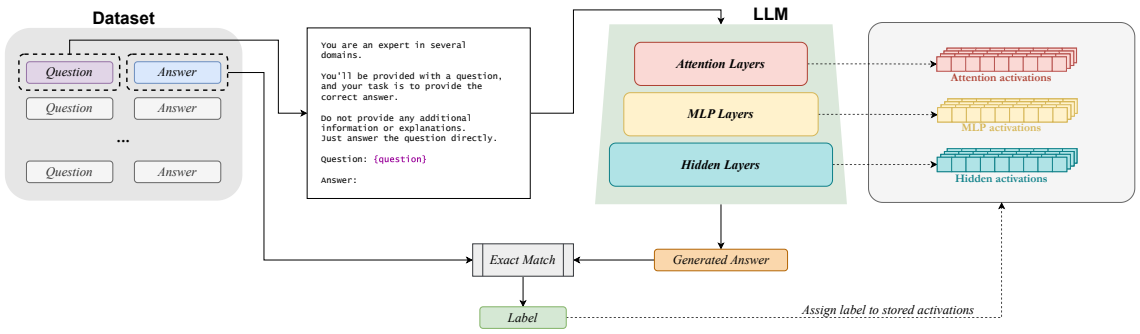

Figure 3.1: Pipeline for extraction and saving of internal LLM activations

3.1.2 Activation Dataset Creation

Once the dataset of statements and their truthfulness labels was prepared, the next step was extracting internal activations from the models Qwen2.5-7B [22], Falcon3-7B-Base [23], Llama-3.1-8B-Instruct [6] and gemma-2-9b-it [21]. Each example is preceded by a prompt that is provided to the language model to guide its response. For the Belief Bank Facts and Belief Bank Constraints datasets, the prompt used is as follows:

> You are an expert in several domains. You'll be provided with a fact, and your task is to confirm or deny its truthfulness. Do not provide any additional information or explanations. Just answer only with yes or no. Is the fact true? Fact: {question}. Answer:

where {question} is replaced with the specific statement to evaluate.
For the Halu Eval dataset, the prompt used is as follows:

> SYS_FACT = You are an expert in several domains. You'll be provided with a fact, and your task is to confirm or deny its truthfulness. Do not provide any additional information or explanations. Just answer only with ÿesör n̈o.̈
>
> USR_FACT = Is the fact true? Fact: question Answer:

Therefore, the LLM receives the prompt followed by the question as input and must respond with "yes" or "no". For each statement in the dataset, the activations corresponding to all layers and components (attention, MLP, hidden states) of the models were recorded. These activations constitute the database on which the various probers for hallucination detection were trained. To verify whether the model responds correctly or hallucinates, a simple matching logic was used between the generated response and the truthfulness label associated with the statement in the dataset: if the actual response (yes or no) is a substring (i.e., is contained) in the model's generated response, then the statement is considered true (non-hallucination); otherwise it is considered false (hallucination).

3.2 LLMs Used

For the experiments conducted in this work, four open-weights models were selected representing the state of the art for their respective sizes and architectural families.

3.2.1 Qwen2.5-7B

Qwen2.5-7B is a model developed by Alibaba Cloud and is part of the Qwen2.5 series. It is a *Decoder-only* Transformer-based model with 7 billion parameters. Compared to its predecessors, Qwen2.5 introduces several architectural optimizations:

- **Grouped Query Attention (GQA)**: Uses GQA instead of standard Multi-Head Attention to optimize Key-Value (KV) cache memory usage during inference, allowing longer contexts and faster generation [1].
- **SwiGLU Activation**: Replaces the classic GeLU activation function with SwiGLU, which has empirically demonstrated improved convergence performance [19].
- **Rotary Positional Embeddings**: Uses rotary positional embeddings to better handle relative token positions and support very large context windows [20].

The model was pre-trained on a massive multilingual corpus, demonstrating excellent capabilities not only in reasoning and language understanding, but also in coding and mathematics.

3.2.2 Falcon3-7B-Base

Falcon3-7B-Base belongs to the family of models developed by the Technology Innovation Institute (TII) of Abu Dhabi. Like Qwen, it is an LLM with 7 billion parameters based on GQA. Falcon3 continues the tradition of the Falcon series by focusing on pre-training data quality (based on the RefinedWeb dataset [17], a rigorously filtered and deduplicated web dataset).

3.2.3 Llama-3.1-8B-Instruct

Llama-3.1-8B-Instruct represents an iteration of the open-weights model family developed by Meta. It has 8 billion parameters and *Instruct* indicates that the model was specifically optimized for dialogue and instruction following through SFT and DPO. Like previous models, it uses GQA to maintain inferential efficiency by reducing the KV cache memory footprint.

3.2.4 Gemma-2-9B-IT

Gemma-2-9B-IT is a model developed by Google DeepMind and is part of the second generation of the Gemma series. With 9 billion parameters, it sits in a slightly higher

dimensional range than classic 7B models, offering reasoning capabilities competitive with much larger models. Unlike Llama and Qwen, Gemma 2 introduces specific architectural novelties that deviate from standard Transformer design:

- **Sliding Window Attention & Global Attention**: The model alternates standard (global) attention layers with *Sliding Window Attention* based layers.
- **Knowledge Distillation**: Instead of classic next-token prediction pre-training from scratch, Gemma-2-9B was trained using Knowledge Distillation techniques from a much larger teacher model, inheriting its reasoning capabilities more efficiently.

Like Llama-3.1-8B-Instruct, Gemma-2-9B-IT was also optimized for dialogue and instruction following through SFT and DPO.

3.3 Preliminary Studies

3.3.1 Hallucination Statistics

Before proceeding with the construction of the universal prober, some statistics were calculated regarding the frequency of hallucinations in the various LLMs for the datasets. Analyzing the model outputs on the datasets, the statistics visible in tables 3.1, 3.2 and 3.3 were calculated.

Table 3.1: Hallucination statistics for Belief Bank Facts

Model	Total statements	Hallucinations	Hallucination Rate
Qwen2.5-7B	27 416	3 565	13.0%
Falcon3-7B-Base	27 416	7 531	27.47%
Llama-3.1-8B-Instruct	27 416	1 799	6.56%
Gemma-2-9B-IT	27 416	802	2.93%

Table 3.2: Hallucination statistics for Belief Bank Constraints

Model	Total statements	Hallucinations	Hallucination Rate
Llama-3.1-8B-Instruct	25 068	14 026	55.95%
Gemma-2-9B-IT	25 756	12 641	49.1%

This is not my mistake, for LLama some activations are missing oh shit, on gandalf? possible?

Table 3.3: Hallucination statistics for Halu Eval

Model	Total statements	Hallucinations	Hallucination Rate
Llama-3.1-8B-Instruct	10 000	2 391	23.91%
Gemma-2-9B-IT	10 000	2 747	27.47%

The results suggest that in general, with the same dataset, models behave similarly, which indicates that hallucinations could be related to intrinsic characteristics in language models as stated by Kalai et al. in [11] (and therefore building a universal prober could be possible).

If we compare results between the three different datasets, we observe that models tend to produce many more *Logical Inconsistencies*

3.3.2 Studio delle singole componenti di layer

Per valutare la capacità discriminativa delle diverse componenti di layer (attention, MLP, hidden states) dei vari LLMS nel rilevamento delle allucinazioni, sono stati condotti esperimenti sistematici su ciascun layer e tipologia di attivazione. In particolare, per ogni layer e componente, sono state estratte le attivazioni corrispondenti e utilizzate come input per addestrare un classificatore Logistic Regression, con l'obiettivo di distinguere tra affermazioni vere e allucinazioni.

La pipeline sperimentale prevede:

1. Estrazione delle attivazioni per ogni layer e componente (attention, MLP, hidden) dai modelli su tutto il dataset.

2. Undersampling e suddivisione del dataset in training e test set, mantenendo la stessa suddivisione per tutti gli esperimenti.

3. Normalizzazione delle attivazioni tramite StandardScaler.

4. Training of a Logistic Regression classifier for each layer/component combination.

5. Evaluation of performance through accuracy metrics.

The results show that some components and layers are particularly effective in discriminating hallucinations, highlighting the presence of specific informative patterns in the internal activations of models

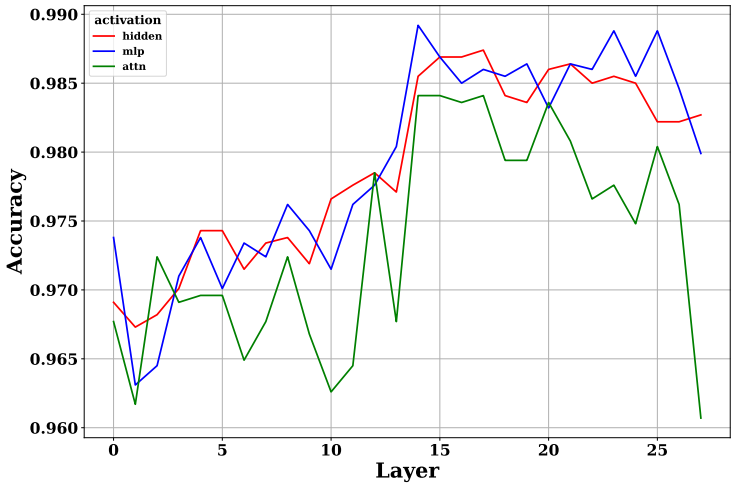

Figure 3.2: Performance of Qwen2.5-7B components on Belief Bank Facts

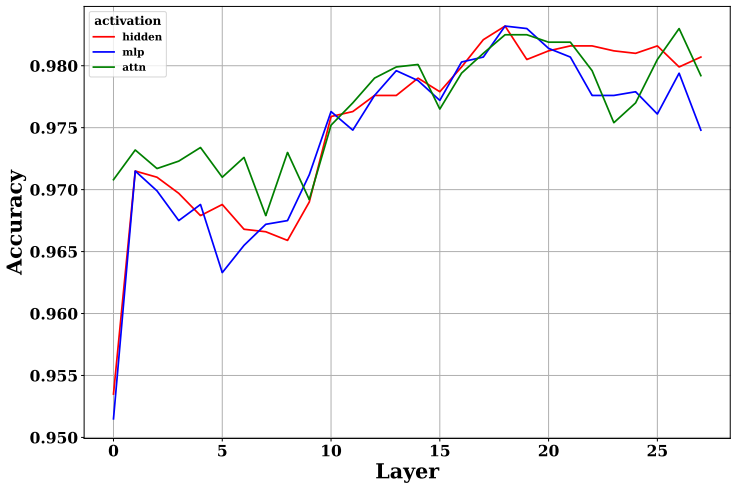

Figure 3.3: Performance of Falcon3-7B-Base components on Belief Bank Facts

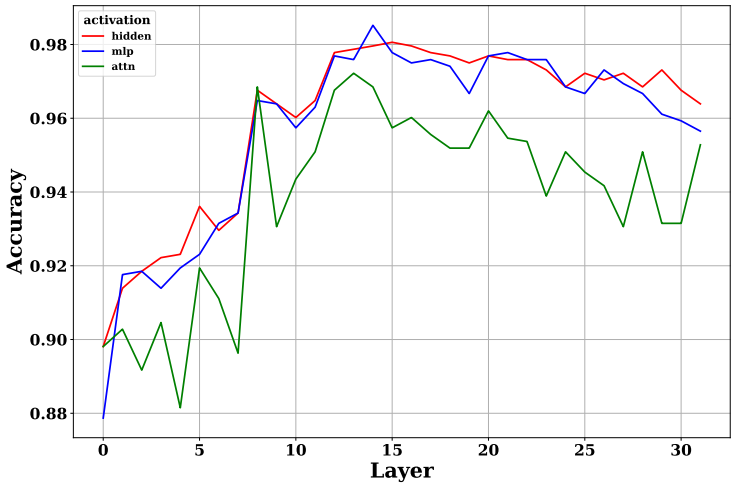

Figure 3.4: Performance of Llama-3.1-8B-Instruct components on Belief Bank Facts

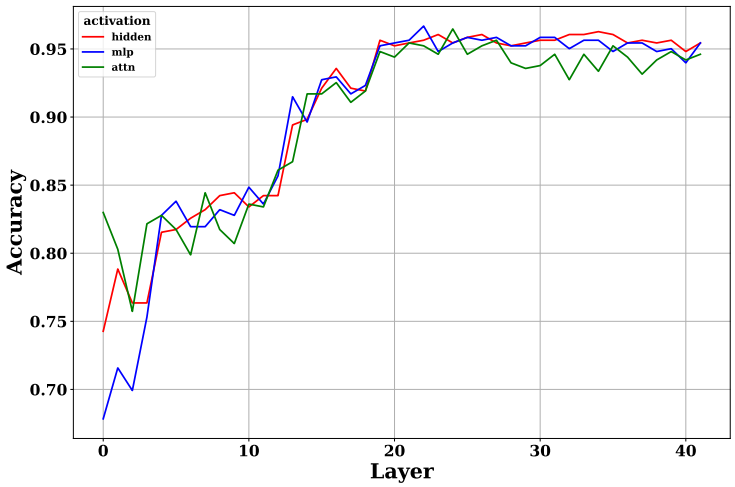

Figure 3.5: Performance of Gemma-2-9B-IT components on Belief Bank Facts

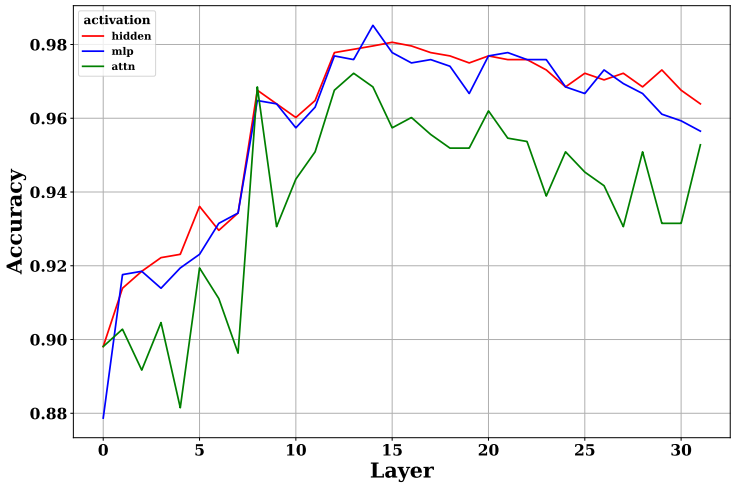

Figure 3.6: Performance of Llama-3.1-8B-Instruct components on Belief Bank Constraints

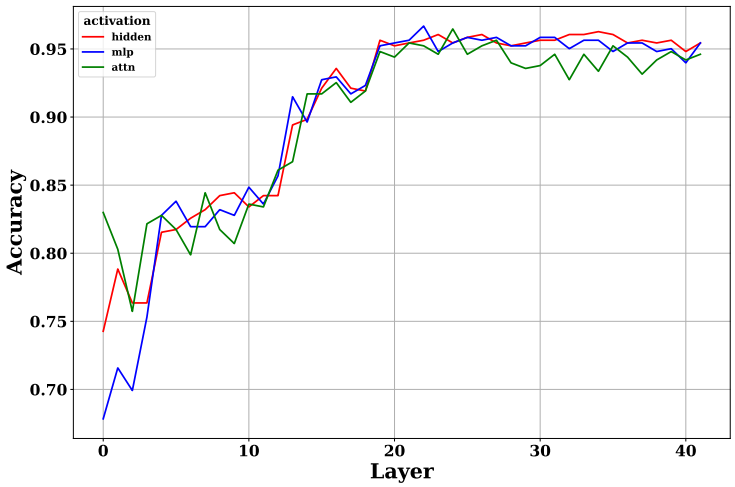

Figure 3.7: Performance of Gemma-2-9B-IT components on Belief Bank Constraints

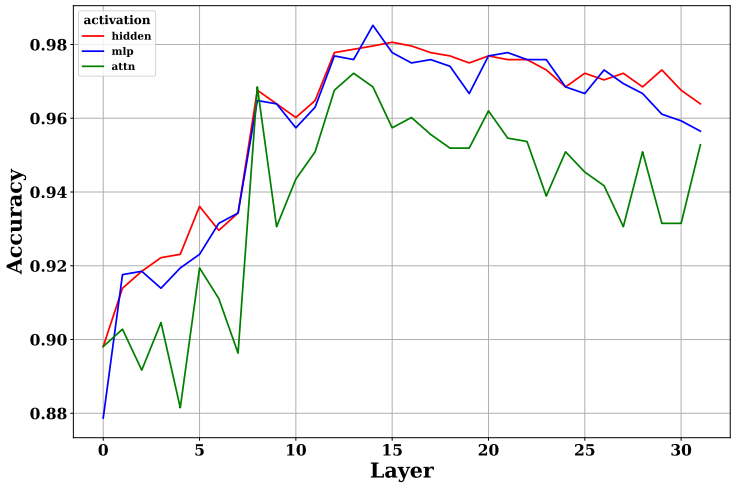

Figure 3.8: Performance of Llama-3.1-8B-Instruct components on Halu Eval

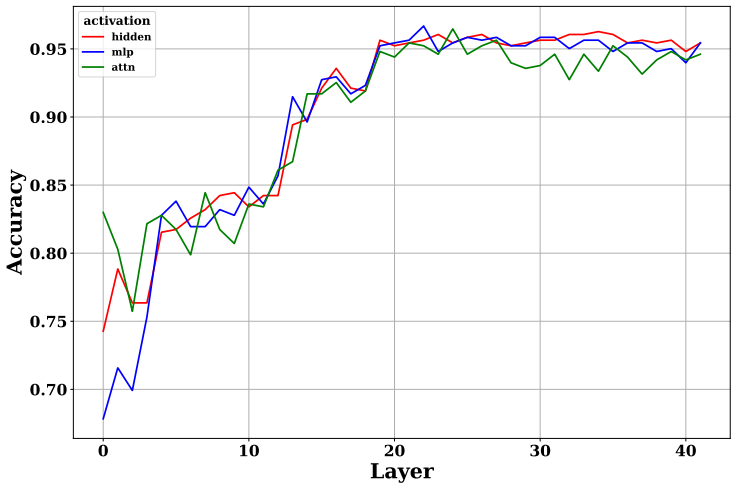

Figure 3.9: Performance of Gemma-2-9B-IT components on Halu Eval

Subsequently, an analysis was performed regarding the distribution of hallucinations and correct responses in the activation space. In order to visualize the activations in a two-dimensional space, the dimensionality reduction technique Principal Component Analysis (PCA) [8] was applied. Some examples follow:

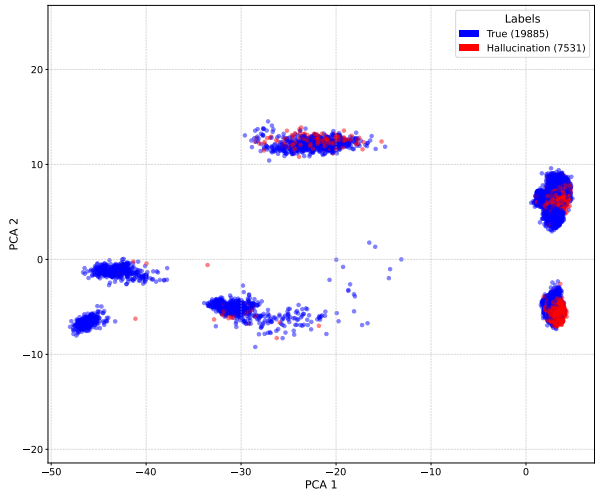

Figure 3.10: PCA of Attention activations of layer 18 of Falcon3-7B-Base

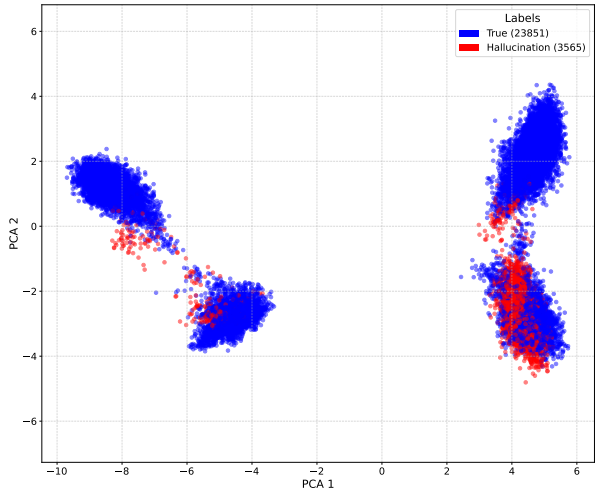

Figure 3.11: PCA of Attention activations of layer 14 of Qwen2.5-7B

For subsequent experiments, the 3 best layers in terms of performance were chosen for each component 3.4, 3.5, 3.6.

Table 3.4: Layers chosen for Belief Bank Facts

Model	Attn	MLP	Hidden
Qwen2.5-7B	14, 15, 17	14, 23, 25	15, 16, 17
Falcon3-7B-Base	18, 19, 26	18, 19, 20	17, 18, 21
Llama-3.1-8B-Instruct	8, 13, 14	14, 15, 21	14, 15, 16
Gemma-2-9B-IT	21, 24, 27	22, 25, 27	23, 26, 34

Table 3.5: Layers chosen for Belief Bank Constraints

Model	Attn	MLP	Hidden
Llama-3.1-8B-Instruct	5, 8, 12	12, 14, 15	13, 14, 15
Gemma-2-9B-IT	23, 27, 33	24, 25, 26	23, 24, 27

Table 3.6: Layers chosen for Halu Eval

Model	Attn	MLP	Hidden
Llama-3.1-8B-Instruct	14, 15, 16	13, 14, 15	14, 15, 16
Gemma-2-9B-IT	21, 26, 27	23, 24, 28	19, 24, 28

We can notice an interesting trend: for LLama, layers between 13 and 16 seem to be the most informative, while for Gemma-2-9B-IT the higher layers (between 23 and 28) are those that offer the best performance. This suggests that the position of the most relevant layers can vary significantly between different models, probably due to architectural differences and specific training processes of each model. However, it can be noted that intermediate layers are the most informative in every model.

3.4 Methodologies for Building the Universal Prober

This section describes the five methodological approaches adopted to build a universal prober capable of detecting hallucinations in language models.

Experiment reproducibility and the use of the same data sets for all methods are guaranteed by setting a common random seed and using deterministic next token sampling in LLMs.

Two dataset management modes are distinguished:

- **Concordant Dataset (for Alignment):** Only examples where Trainer and Tester agree on classification (same label) are selected. Undersampling is applied to these to balance classes. This is essential for learning a coherent mapping between tester and trainer
  I'm writing here, but it's just for reference: I was thinking, can we plot a before and after alignment of activations? to understand if somehow they get closer? Appendix B

- **Independent Dataset (for Probing/Autoencoder/Encoder):** Undersampling is applied to balance classes specifically on the labels of the model in question (Trainer or Tester), independently from the other model. This maximizes the amount of data available for training robust models.

3.4.1 FullLinear Approach

The first approch for building the universal prober, a linear approach was adopted. In this scenario, the internal activations of the models, extracted from the selected layers and components, were used to train and evaluate a Logistic Regression classifier. The objective is to verify performance using only linear methods.

Experimental Pipeline

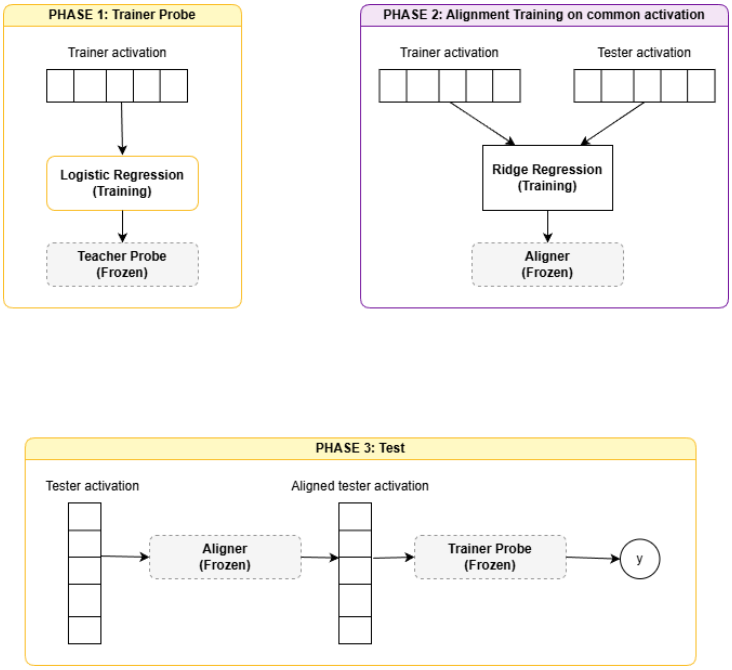

Figure 3.12: FullLinear experimental pipeline

The experimental pipeline in figure 3.12 consists of training a Logistic Regression classifier on the activation space of the trainer model to detect hallucinations, followed by evaluating its performance through accuracy metrics. Subsequently, a linear alignment is performed between the latent space of the tester model and that of the trainer model via a linear projection (Ridge Regression) trained on a subset of concordant data. Finally, the classifier's ability to discriminate hallucinations on tester model data after alignment is evaluated.

For the Logistic Regressor, the following hyperparameters were used:

- **solver**: `lbfgs`
- **max_iterations**: 10000
- **class_weight:** `balanced`

3.4.2 AdapterMLP Approach

In this approach, a non-linear adaptation procedure of the tester model's latent space towards the trainer model's space is explored using a low-dimensional alignment network (called AlignmentNetwork or AdapterMLP). The objective is to observe whether introducing a non-linear component in the alignment improves universal prober performance, while maintaining a linear classifier (Logistic Regression) on the trainer model.

Architecture and Loss

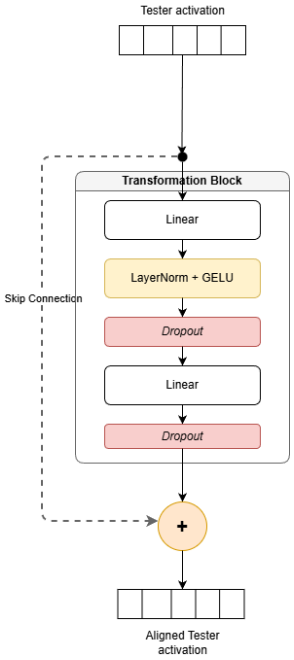

Figure 3.13: AlignmentNetwork architecture of the AdapterMLP approach

Some key characteristics of the AlignmentNetwork include:

- **Zero-init:** the last layers of the decompression are initialized to zero so that the network starts close to a linear identity transformation, allowing the non-linear component to emerge only if truly useful.
- **MixedLoss:** the loss function combines Mean Squared Error (MSE) and a component based on cosine similarity:

$$\mathcal{L} = \alpha \cdot \text{MSE}(\hat{y}, y) + \beta \cdot (1 - \text{cosine\_sim}(\hat{y}, y))$$

  with configurable weights α (MSE) and β (cosine) to balance fidelity and angular alignment.

where MSE is defined as:

$$\text{MSE}(\hat{y}, y) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

and cosine similarity is defined as:

$$\text{cosine\_sim}(\hat{y}, y) = \frac{\hat{y} \cdot y}{\|\hat{y}\|\|y\|} = \frac{\sum_{i=1}^{N} \hat{y}_i y_i}{\sqrt{\sum_{i=1}^{N} \hat{y}_i^2}\sqrt{\sum_{i=1}^{N} y_i^2}}$$

Experimental Pipeline

The experimental pipeline in figure 3.14 first involves training a probe (Logistic Regression) on the trainer's activation space using the entire balanced trainer training set; this classifier will remain fixed. Subsequently, the AlignmentNetwork is trained to project tester activations into the trainer's space, minimizing the loss between projected activations and original trainer activations using the balanced concordant activations dataset. Finally, the tester's test activations are projected via the AlignmentNetwork and classified using the probe trained on the trainer.

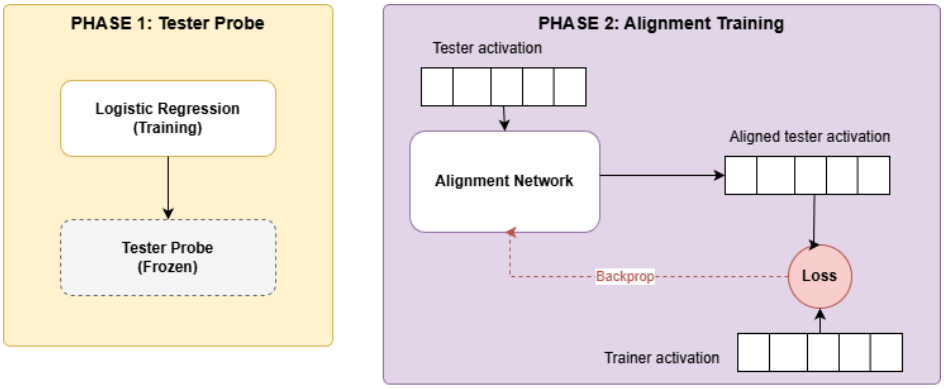

Figure 3.14: AdapterMLP approach experimental pipeline

For the Logistic Regressor, the following hyperparameters were used:

- **solver**: `lbfgs`
- **max_iterations**: 10000
- **class_weight:** `balanced`

In table 3.7, the main hyperparameters used for training the AlignmentNetwork are reported:

Table 3.7: AlignmentNetwork hyperparameter values apply to all combinations of Trainer, Tester and Layer Type.

Parameter	Value
Hidden Dimension	128
Dropout	0.5
Learning Rate (LR)	1e-3
Weight Decay	1e-1
Batch Size	32
Early Stopping δ	1e-4
Gradient Clipping	1.0
Optimizer	AdamW
Scheduler	CosineAnnealingLR
α (Loss Weight)	0.01
β (Loss Weight)	1.0

3.4.3 Full Non-linear Approach

This approach follows the previous one regarding the alignment part between the two LLMs. The difference lies in the non-linear prober (MLP) as classifier on the trainer. The objective is to verify performance using both non-linear components following the same experimental pipeline.

Architecture, Components and Loss

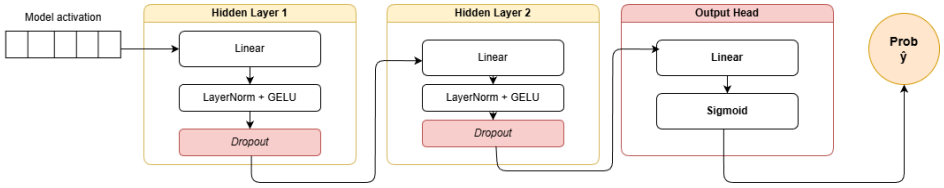

Figure 3.15: MLP Prober architecture of the full non-linear approach

- **AlignmentNetwork:** The same architecture described in the previous approach is used with the same loss
- **MLP Prober in figure 3.15:** Non-linear classifier for hallucination detection: fully-connected network with normalized intermediate layers (LayerNorm), GELU activation and dropout.
- **BCEWithLogitsLoss:** combines a sigmoid function and Binary Cross Entropy for numerical stability.

BCEWithLogitsLoss is defined as:

$$\mathcal{L}(x, y) = -\left(y \cdot \log(\sigma(x)) + (1 - y) \cdot \log(1 - \sigma(x))\right)$$

where $\sigma(x)$ is the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Experimental Pipeline

The experimental pipeline in figure 3.16 is therefore identical to that of the AdapterMLP approach, with the difference that the classifier on the trainer is now a non-linear MLP Prober.

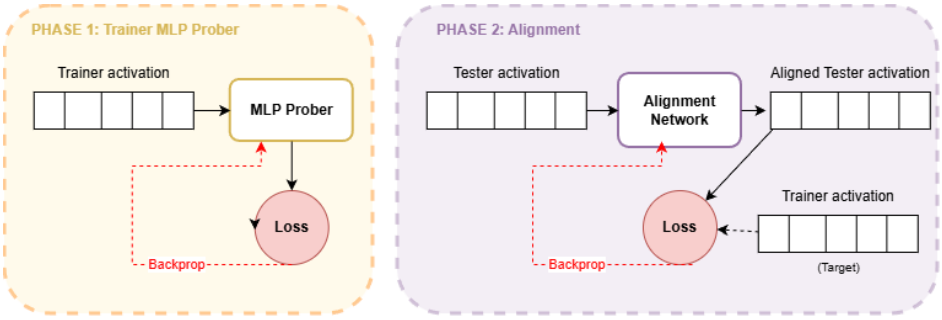

Figure 3.16: Full non-linear approach experimental pipeline

In tables 3.8 and 3.9, the main hyperparameters used for training the AlignmentNetwork and MLP Prober are reported:

Table 3.8: AlignmentNetwork hyperparameters for the full non-linear approach.

Parameter	**Value**
Alignment Network	
Hidden Dimension	128
Dropout	0.5
Learning Rate	1e-3
Weight Decay	1e-1
Batch Size	32
Early Stopping Patience	50
Loss Function	
α (Reconstruction)	0.01
β (Contrastive)	1.0

Table 3.9: MLP Prober hyperparameters for the full non-linear approach.

Parameter	Value
Hidden Dimension	64
Dropout	0.5
Learning Rate	1e-3
Weight Decay	1e-2
Batch Size	64
Early Stopping Patience	30
Optimizer	AdamW
Scheduler	CosineAnnealingLR

3.4.4 Reduced Non-linear Approach

This approach reduces activation dimensionality via autoencoder before alignment.

Architecture and Main Components

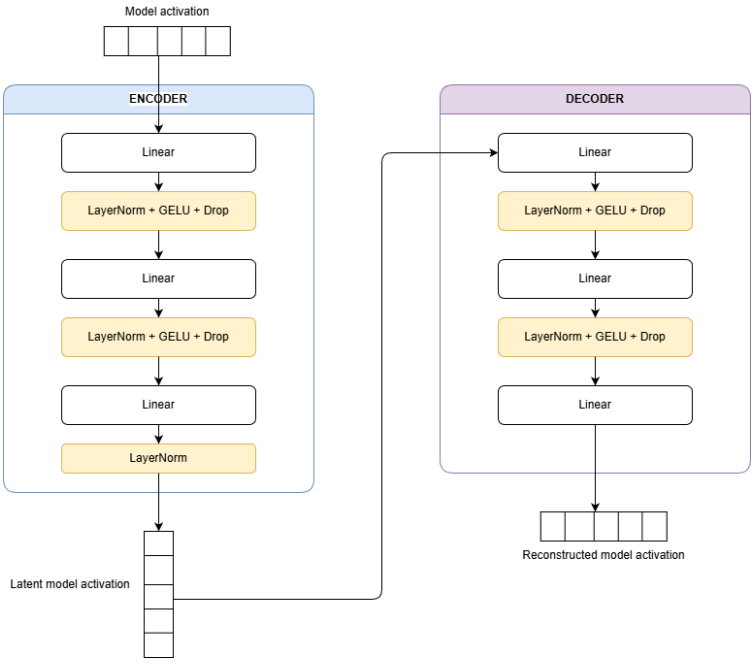

Figure 3.17: Autoencoder architecture of the reduced non-linear approach

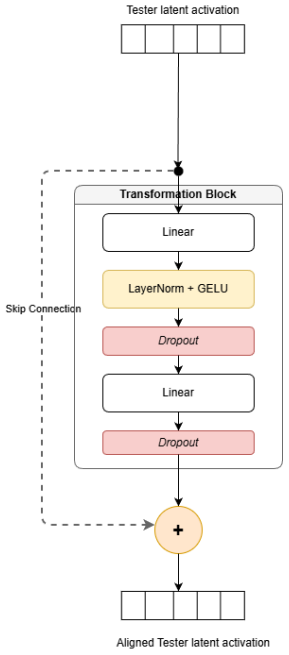

Figure 3.19: AlignmentNetwork architecture of the reduced non-linear approach

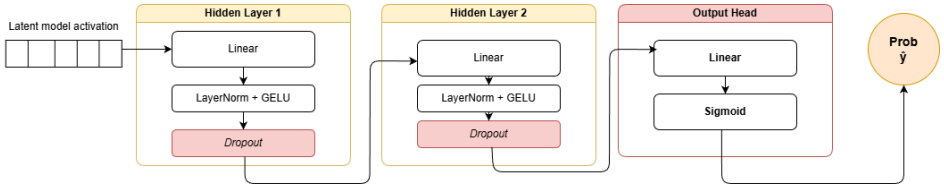

Figure 3.18: MLP Prober architecture of the reduced non-linear approach

- **Autoencoder (Trainer & Tester) in figure 3.17**: a separate autoencoder is trained on the trainer's activation space and one on the tester's. The encoders produce latent representations of common dimension. The loss used is Mean Squared Error (MSE).
- **Alignment Network (latent) in figure 3.19**: Maps the tester's latent space to the trainer's.
- **MLP Prober in figure 3.18**: Classifies operating on the reduced latent space of the trainer.

Experimental Pipeline

The experimental pipeline in figure 3.20 first involves training two autoencoders: one on the trainer's activations and the other on the tester's, thus obtaining two distinct latent spaces. Subsequently, the activations are encoded in their respective latent

spaces. An MLP classifier is trained on the trainer's latent space. Then, an Alignment Network is trained that maps the tester's latent representations to the trainer's latent space using concordant data. Finally, for evaluation, the MLP classifier trained on the trainer is used to classify the tester's representations after alignment and dimensionality reduction.

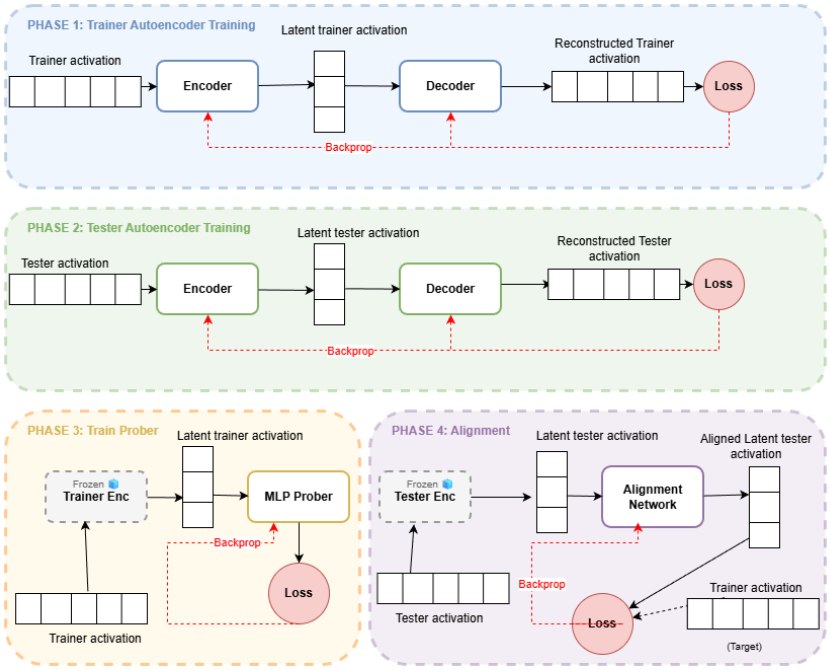

Figure 3.20: Reduced non-linear approach experimental pipeline

In tables 3.10, 3.11 and 3.12, the main updated hyperparameters used in this approach are reported:

Table 3.10: AutoEncoder hyperparameters (Trainer and Tester).

Parameter	**Value**
Latent Dimension	128
Hidden Dimension	256
Dropout	0.2
Learning Rate	1e-3
Weight Decay	1e-2
Batch Size	64
Loss	MSELoss

Table 3.11: AlignmentNetwork hyperparameters for the reduced approach.

Parameter	**Value**
Network Parameters	
Hidden Dimension	256
Dropout	0.3
Learning Rate	1e-3
Weight Decay	1e-2
Batch Size	32
Loss Weights	
α (Reconstruction)	0.5
β (Contrastive)	0.5

Table 3.12: MLP Prober hyperparameters (on latent space).

Parameter	**Value**
Hidden Dimension	64
Dropout	0.3
Learning Rate	1e-3
Batch Size	64
Loss	BCEWithLogitsLoss

3.4.5 One-For-All Approach (Frozen Head)

In this approach, a two-phase procedure is adopted, designed to evaluate the transferability of a decision function (the classification *head*) between two different models while keeping the same head and adapting only the tester's encoder. Unlike previous approaches, this method does not require concordant data for alignment but uses independent balanced datasets for each model.

Architecture and main components

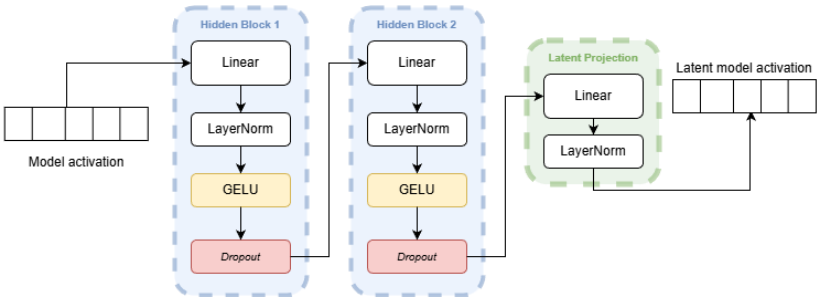

Figure 3.21: One-For-All encoder architecture

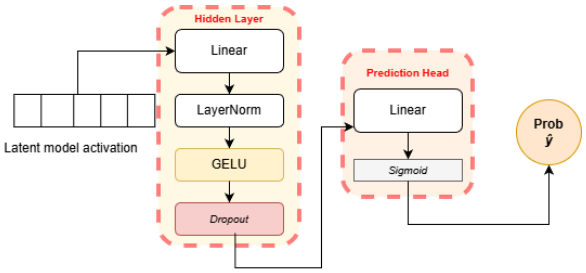

Figure 3.22: One-For-All Classification Head architecture

- **Encoder in Figure 3.21:** multi-block feed-forward network that maps the input space (activation dimension) to a reduced-dimension latent space (256).
- **Classification Head in Figure 3.22:** compact module that takes the latent vector as input and returns a binary logit. Implemented as a small MLP.

Experimental pipeline

The experimental pipeline in Figure 3.23 first involves joint training of the encoder and classification head on the trainer model's activation space (End-to-End). Once training is complete, the head is frozen (*Frozen Head*) to keep the learned decision function unchanged. Subsequently, a new encoder is trained on the tester model's activation space: the output of this new encoder is passed to the trainer's frozen head, and the loss is calculated directly on the classification error (BCEWithLogitsLoss). In this way, the tester learns to produce latent representations compatible with the trainer's "head".

The classification head is trained only once with the trainer, then frozen and reused for the tester. So yes: if reused on the trainer it maintains the same results perfect, from the figure I understood it was re-trained, sorry

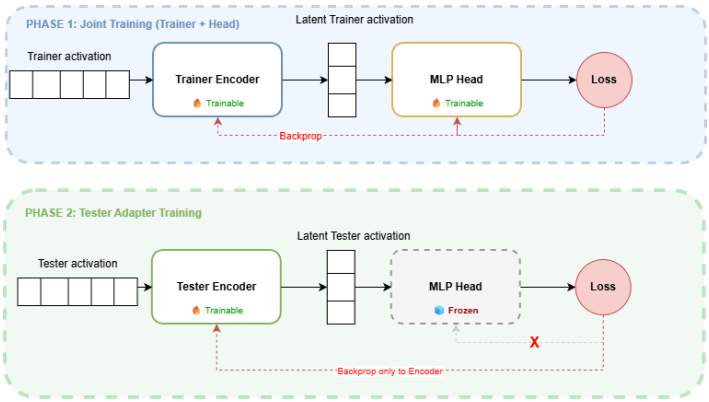

Figure 3.23: One-For-All experimental pipeline

Tables 3.13 and 3.14 report the main hyperparameters used for training the Encoders and Classification Head:

Table 3.13: Encoder hyperparameters (Trainer and Tester Adapter).

Parameter	**Value**
Latent Dimension	256
Hidden Dimension	512
Dropout	0.3
Learning Rate	1e-3
Weight Decay	1e-2
Batch Size	64
Early Stopping Patience	15
Optimizer	AdamW

Table 3.14: Classification Head hyperparameters (Shared/Frozen).

Parameter	**Value**
Latent Dimension (Input)	256
Hidden Dimension	128
Dropout	0.3
Learning Rate	1e-3
Batch Size	64

Chapter 4

Results

In this chapter, the results obtained from experiments conducted to evaluate the effectiveness of the universal prober in detecting hallucinations in LLMs are presented. In particular, a table is reported for each (trainer-tester)-Dataset pair to analyze the best approach for each combination.

4.1 Falcon3-7B-Base e Qwen2.5-7B su BeliefBank-Facts

Table 4.1: Results for pair (Falcon3-7B-Base, Qwen2.5-7B) on BeliefBankFacts

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
Qwen2.5-7B	Falcon3-7B-Base	FullLinear	attn	0.987	0.962	0.999	0.992
Qwen2.5-7B	Falcon3-7B-Base	FullLinear	mlp	0.986	0.967	0.999	0.987
Qwen2.5-7B	Falcon3-7B-Base	FullLinear	hidden	0.985	0.967	0.999	0.990
Falcon3-7B-Base	Qwen2.5-7B	FullLinear	attn	0.987	0.958	0.997	0.989
Falcon3-7B-Base	Qwen2.5-7B	FullLinear	mlp	0.983	0.961	0.996	0.989
Falcon3-7B-Base	Qwen2.5-7B	FullLinear	hidden	0.983	0.959	0.996	0.984
Qwen2.5-7B	Falcon3-7B-Base	AdapterMLP	attn	0.987	0.968	0.999	0.984
Qwen2.5-7B	Falcon3-7B-Base	AdapterMLP	mlp	0.986	0.969	0.999	0.986
Qwen2.5-7B	Falcon3-7B-Base	AdapterMLP	hidden	0.985	0.968	0.999	0.990
Falcon3-7B-Base	Qwen2.5-7B	AdapterMLP	attn	0.987	0.848	0.997	0.912
Falcon3-7B-Base	Qwen2.5-7B	AdapterMLP	mlp	0.983	0.913	0.996	0.967
Falcon3-7B-Base	Qwen2.5-7B	AdapterMLP	hidden	0.983	0.889	0.996	0.922
Qwen2.5-7B	Falcon3-7B-Base	FullNonLinear	attn	0.993	0.981	1.000	0.994
Qwen2.5-7B	Falcon3-7B-Base	FullNonLinear	mlp	0.994	0.977	1.000	0.994
Qwen2.5-7B	Falcon3-7B-Base	FullNonLinear	hidden	0.992	0.981	1.000	0.996
Falcon3-7B-Base	Qwen2.5-7B	FullNonLinear	attn	0.992	0.890	0.999	0.938
Falcon3-7B-Base	Qwen2.5-7B	FullNonLinear	mlp	0.988	0.941	0.999	0.988
Falcon3-7B-Base	Qwen2.5-7B	FullNonLinear	hidden	0.991	0.931	0.999	0.985
Qwen2.5-7B	Falcon3-7B-Base	ReducedNonLinear	attn	0.9916	0.9706	0.9994	0.9953
Qwen2.5-7B	Falcon3-7B-Base	ReducedNonLinear	mlp	0.9897	0.9608	0.9997	0.9915
Qwen2.5-7B	Falcon3-7B-Base	ReducedNonLinear	hidden	0.9935	0.9586	0.9997	0.9892
Falcon3-7B-Base	Qwen2.5-7B	ReducedNonLinear	attn	0.9841	0.8537	0.9983	0.9367
Falcon3-7B-Base	Qwen2.5-7B	ReducedNonLinear	mlp	0.9807	0.9411	0.9971	0.9832
Falcon3-7B-Base	Qwen2.5-7B	ReducedNonLinear	hidden	0.9770	0.9004	0.9952	0.9503
Qwen2.5-7B	Falcon3-7B-Base	One-For-All	attn	0.9892	0.9916	0.9995	0.9992
Qwen2.5-7B	Falcon3-7B-Base	One-For-All	mlp	0.9860	0.9858	0.9993	0.9980
Qwen2.5-7B	Falcon3-7B-Base	One-For-All	hidden	0.9906	0.9898	0.9998	0.9991
Falcon3-7B-Base	Qwen2.5-7B	One-For-All	attn	0.9905	0.9921	0.9985	0.9997
Falcon3-7B-Base	Qwen2.5-7B	One-For-All	mlp	0.9869	0.9874	0.9985	0.9997
Falcon3-7B-Base	Qwen2.5-7B	One-For-All	hidden	0.9889	0.9949	0.9988	0.9999

4.2 LLama-3.1-8B-Instruct e Gemma-2-9B-IT su BeliefBankFacts

Table 4.2: Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on BeliefBankFacts

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	attn	0.965	0.919	0.986	0.974
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	mlp	0.963	0.928	0.988	0.977
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	hidden	0.959	0.941	0.988	0.982
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	attn	0.982	0.959	0.996	0.986
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	mlp	0.973	0.948	0.997	0.983
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	hidden	0.969	0.946	0.996	0.983
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	attn	0.965	0.901	0.986	0.961
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	mlp	0.9627	0.9074	0.9883	0.9624
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	hidden	0.9585	0.9204	0.9878	0.9752
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	attn	0.982	0.938	0.996	0.974
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	mlp	0.973	0.936	0.997	0.975
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	hidden	0.969	0.934	0.996	0.975
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	attn	0.975	0.938	0.994	0.976
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	mlp	0.954	0.917	0.990	0.967
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	hidden	0.961	0.929	0.987	0.969
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	attn	0.984	0.950	0.998	0.982
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	mlp	0.986	0.961	0.998	0.983
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	hidden	0.981	0.954	0.998	0.984
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	attn	0.963	0.900	0.993	0.970
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	mlp	0.952	0.888	0.988	0.955
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	hidden	0.940	0.904	0.988	0.959
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	attn	0.986	0.932	0.999	0.983
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	mlp	0.970	0.923	0.995	0.971
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	hidden	0.964	0.903	0.994	0.969
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	attn	0.9627	0.9861	0.9923	0.9972
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	mlp	0.9564	0.9843	0.9820	0.9967
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	hidden	0.9502	0.9843	0.9898	0.9976
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	attn	0.9861	0.9627	0.9982	0.9894
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	mlp	0.9852	0.9523	0.9984	0.9874
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	hidden	0.9778	0.9606	0.9950	0.9891

4.3 LLama-3.1-8B-Instruct e Gemma-2-9B-IT su BeliefBankCalibration

Table 4.3: Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on BeliefBankCalibration

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	attn	0.9713	0.9665	0.9950	0.9926
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	mlp	0.9727	0.9725	0.9953	0.9946
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	hidden	0.9744	0.9742	0.9953	0.9949
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	attn	0.9697	0.9715	0.9955	0.9936
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	mlp	0.9730	0.9698	0.9958	0.9933
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	hidden	0.9736	0.9699	0.9960	0.9936
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	attn	0.9713	0.7972	0.9950	0.8789
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	mlp	0.9727	0.8723	0.9953	0.9352
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	hidden	0.9744	0.8790	0.9953	0.9440
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	attn	0.9697	0.8820	0.9955	0.9424
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	mlp	0.9730	0.8982	0.9958	0.9554
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	hidden	0.9736	0.8903	0.9960	0.9521
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	attn	0.9848	0.8541	0.9982	0.9093
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	mlp	0.9842	0.9016	0.9984	0.9397
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	hidden	0.9856	0.9042	0.9986	0.9510
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	attn	0.9893	0.8991	0.9990	0.9512
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	mlp	0.9829	0.9395	0.9984	0.9756
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	hidden	0.9848	0.9111	0.9983	0.9546
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	attn	0.9713	0.9096	0.9955	0.9786
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	mlp	0.9680	0.8485	0.9950	0.9544
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	hidden	0.9719	0.8367	0.9959	0.9687
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	attn	0.9673	0.8816	0.9958	0.9683
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	mlp	0.9679	0.9279	0.9942	0.9785
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	hidden	0.9651	0.9438	0.9938	0.9824
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	attn	0.9843	0.9885	0.9985	0.9986
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	attn	0.9878	0.9844	0.9989	0.9979
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	mlp	0.9847	0.9807	0.9980	0.9971
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	mlp	0.9793	0.9855	0.9978	0.9986
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	hidden	0.9873	0.9828	0.9984	0.9957
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	hidden	0.9840	0.9838	0.9985	0.9983

4.4 LLama-3.1-8B-Instruct e Gemma-2-9B-IT su HaluEval

Table 4.4: Results for pair (LLama-3.1-8B-Instruct, Gemma-2-9B-IT) on HaluEval

Trainer	Tester	Approach	Type	AccTr	AccTe	AurocTr	AurocTe
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	attn	0.6962	0.7884	0.7670	0.8525
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	mlp	0.7010	0.7799	0.7582	0.8559
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullLinear	hidden	0.6955	0.7647	0.7715	0.8380
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	attn	0.7568	0.7540	0.8231	0.8161
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	mlp	0.7599	0.7519	0.8297	0.8218
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullLinear	hidden	0.7508	0.7408	0.8173	0.8138
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	attn	0.6962	0.7508	0.7670	0.8113
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	mlp	0.7010	0.7483	0.7582	0.8133
gemma-2-9b-it	Llama-3.1-8B-Instruct	AdapterMLP	hidden	0.6955	0.7544	0.7715	0.8199
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	attn	0.7568	0.7157	0.8231	0.7680
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	mlp	0.7599	0.7185	0.8297	0.7908
Llama-3.1-8B-Instruct	gemma-2-9b-it	AdapterMLP	hidden	0.7508	0.7352	0.8173	0.7882
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	attn	0.7226	0.7605	0.7897	0.8335
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	mlp	0.7310	0.7677	0.7968	0.8417
gemma-2-9b-it	Llama-3.1-8B-Instruct	FullNonLinear	hidden	0.7178	0.7647	0.7934	0.8350
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	attn	0.7799	0.7247	0.8528	0.7906
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	mlp	0.7720	0.7415	0.8515	0.7952
Llama-3.1-8B-Instruct	gemma-2-9b-it	FullNonLinear	hidden	0.7708	0.7463	0.8470	0.8099
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	attn	0.7038	0.7053	0.7720	0.7883
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	mlp	0.7010	0.6695	0.7832	0.7445
gemma-2-9b-it	Llama-3.1-8B-Instruct	ReducedNonLinear	hidden	0.6941	0.6956	0.7663	0.7729
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	attn	0.7404	0.6509	0.8293	0.7177
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	mlp	0.7544	0.6272	0.8309	0.6985
Llama-3.1-8B-Instruct	gemma-2-9b-it	ReducedNonLinear	hidden	0.7465	0.6523	0.8354	0.7145
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	attn	0.7094	0.7762	0.7683	0.8553
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	attn	0.7793	0.7010	0.8502	0.7680
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	mlp	0.7066	0.7629	0.7728	0.8445
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	mlp	0.7696	0.7045	0.8492	0.7666
gemma-2-9b-it	Llama-3.1-8B-Instruct	One-For-All	hidden	0.7129	0.7768	0.7715	0.8484
Llama-3.1-8B-Instruct	gemma-2-9b-it	One-For-All	hidden	0.7574	0.7052	0.8371	0.7714

Chapter 5

Discussion

This chapter discusses the results obtained from the experiments conducted to evaluate the different approaches proposed for hallucination detection in LLMs. The main objective is to analyze the effectiveness of transferring detection capabilities from a "Trainer" model to a "Tester" model through different alignment techniques.

5.1 Comparison Between Methods

The results show that the suggested transferability methods are fairly equivalent to each other, presenting very similar metrics on both the tester and trainer. The study therefore demonstrates that it is possible to transfer probers for hallucination detection between different models, especially for simple factual verification tasks. The *One-For-All* approach proves to be the most promising method for building a "universal prober".

5.2 Dataset Analysis

The results highlight a clear distinction between datasets based on simple facts (BeliefBank) and more complex ones (HaluEval).

- **BeliefBankFacts and BeliefBankCalibration**: On these datasets, all models and approaches achieve very high performance, with accuracies often exceeding 95%. This suggests that the distinction between true and false statements about general knowledge facts is encoded very clearly and linearly separable in the activation spaces of LLMs. Transferability is therefore high.

- **HaluEval**: This dataset proves to be significantly more difficult. Accuracies drop to around 70-80% for all approaches. In this scenario, the *FullLinear* often clearly outperforms the transfer approaches. This indicates that the "directions" encoding hallucinations in more complex contexts are much more model-specific and less transferable through the tested alignment techniques.

5.3 Model Analysis

The comparison between model pairs offers further insights:

- **Falcon3-7B-Base and Qwen2.5-7B**: This pair shows exceptional compatibility on BeliefBankFacts, with nearly perfect transfer results. This could indicate a structural or pre-training similarity that facilitates the alignment of their latent spaces.
- **Llama-3.1-8B-Instruct and Gemma-2-9B-IT**: This pair also performs well on BeliefBank, but shows greater difficulties on HaluEval. It is interesting to note that in some cases (e.g., HaluEval), the "tester" outperforms the "trainer", suggesting that some internal representations may be more suitable for hallucination detection in complex contexts.

5.4 Layer Analysis

Regarding the layer type (*attn*, *mlp*, *hidden*), no absolute winner emerges across all scenarios. However, it is often noted that *attn* (attention output) and *hidden* (hidden states) activations tend to provide slightly more stable performance compared to *mlp*. The *One-For-All* approach nevertheless demonstrates remarkable robustness regardless of the layer type used, managing to extract useful information from all internal components analyzed.

Chapter 6

Cross-Domain Studies

To further evaluate the generalizability of transferred probers, cross-domain experiments were conducted. In this context, the *One-For-All* approach was evaluated by training the encoder on a source dataset ("Train Set") and applying it to project the activations of the **same source dataset**, but classifying them using the Teacher's Head specific to a different target dataset ("Head Set"). This scenario evaluates the **universality of the latent space**: it tests whether the representation of truth learned in one domain is compatible with the decision boundary learned in another domain.

The results, summarized in Table 6.1, show that the encoding learned via encoder on simple facts datasets (BeliefBank) transfers with exceptional effectiveness even to the classification head learned on a more complex dataset (HaluEval), allowing the Tester to achieve performance nearly identical to the Trainer (often >99% accuracy). Conversely, the encoding learned on HaluEval, while achieving good results, shows a performance drop when transferred to BeliefBank (approximately 87–90% accuracy), suggesting that the latent space structure learned on complex tasks may be less "universal" or noisier compared to that learned on simple facts.

Align Train	Target/Head	Teacher	Student	Layer	Acc (Tr)	AUROC (Tr)	Acc (Te)	AUROC (Te)
BBC	BBF	Gemma	Llama	attn	0.991	0.999	0.995	0.999
BBC	BBF	Gemma	Llama	mlp	0.992	0.998	0.984	0.995
BBC	BBF	Gemma	Llama	hidden	0.995	0.997	0.990	0.999
BBC	BBF	Llama	Gemma	attn	0.991	0.996	0.994	0.999
BBC	BBF	Llama	Gemma	mlp	0.980	0.995	0.994	1.000
BBC	BBF	Llama	Gemma	hidden	0.993	0.999	0.993	0.999
BBC	HE	Gemma	Llama	attn	0.991	0.997	0.995	0.999
BBC	HE	Gemma	Llama	mlp	0.993	0.997	0.984	0.993
BBC	HE	Gemma	Llama	hidden	0.995	0.996	0.991	0.999
BBC	HE	Llama	Gemma	attn	0.993	0.998	0.994	0.999
BBC	HE	Llama	Gemma	mlp	0.980	0.990	0.994	1.000
BBC	HE	Llama	Gemma	hidden	0.993	0.998	0.993	0.999
BBF	BBC	Gemma	Llama	attn	0.986	0.997	0.994	0.997
BBF	BBC	Gemma	Llama	mlp	0.983	0.992	0.994	0.998
BBF	BBC	Gemma	Llama	hidden	0.972	0.995	0.993	0.998
BBF	BBC	Llama	Gemma	attn	0.994	0.997	0.987	0.994
BBF	BBC	Llama	Gemma	mlp	0.992	0.999	0.983	0.987
BBF	BBC	Llama	Gemma	hidden	0.992	0.999	0.977	0.986
BBF	HE	Gemma	Llama	attn	0.985	0.993	0.995	0.997
BBF	HE	Gemma	Llama	mlp	0.984	0.989	0.994	0.998
BBF	HE	Gemma	Llama	hidden	0.971	0.991	0.993	0.997
BBF	HE	Llama	Gemma	attn	0.994	0.998	0.986	0.997
BBF	HE	Llama	Gemma	mlp	0.992	1.000	0.983	0.996
BBF	HE	Llama	Gemma	hidden	0.991	0.996	0.979	0.996
HE	BBC	Gemma	Llama	attn	0.872	0.916	0.907	0.924
HE	BBC	Gemma	Llama	mlp	0.873	0.928	0.905	0.950
HE	BBC	Gemma	Llama	hidden	0.871	0.926	0.904	0.907
HE	BBC	Llama	Gemma	attn	0.909	0.941	0.884	0.894
HE	BBC	Llama	Gemma	mlp	0.900	0.944	0.871	0.897
HE	BBC	Llama	Gemma	hidden	0.893	0.935	0.880	0.928
HE	BBF	Gemma	Llama	attn	0.877	0.925	0.906	0.950
HE	BBF	Gemma	Llama	mlp	0.872	0.924	0.903	0.945
HE	BBF	Gemma	Llama	hidden	0.872	0.928	0.905	0.940
HE	BBF	Llama	Gemma	attn	0.907	0.948	0.885	0.932
HE	BBF	Llama	Gemma	mlp	0.901	0.950	0.871	0.928
HE	BBF	Llama	Gemma	hidden	0.897	0.935	0.880	0.929

Table 6.1: Cross-Domain Results (One-For-All). Tr=Trainer, Te=Tester, BBC=BeliefBankConstraints, BBF=BeliefBankFacts, HE=HaluEval

Chapter 7

Conclusions

This chapter summarizes the main results obtained from this work and future perspectives.

7.1 Summary of Results

This work explored the feasibility of a "universal prober" for hallucination detection in LLMs, evaluating different techniques for aligning latent spaces between "Trainer" and "Tester" models. The experiments conducted on heterogeneous model pairs (Llama-3, Gemma-2, Falcon-3, and Qwen-2.5) and on datasets of varying complexity (BeliefBank and HaluEval) led to the following key results:

- **Transfer Effectiveness**: The proposed transferability methods proved effective, especially for simple factual verification tasks such as those in BeliefBank, where the performance of the transferred prober was comparable to that of dedicated classifiers.
- **Asymmetry in Generalization**: Cross-domain experiments revealed an interesting asymmetry: the encoding learned on "simple" datasets (BeliefBank) transfers effectively to complex contexts (HaluEval), while the reverse is less performant. This suggests that latent structures of truthfulness are more clearly defined and "universal" in basic factual tasks.
- **Robustness of the One-For-All Approach**: The One-For-All approach proved to be the most promising method for building a universal prober, showing robustness regardless of the type of layer analyzed (attn, mlp, hidden).

7.2 Future Work

This work can be continued in several directions:

- **Multimodal Experiments**: Extend the approach to multimodal models that integrate text and images.

Bibliography

[1] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

[2] Amos Azaria and Tom Mitchell. The internal state of an llm knows when it's lying. *arXiv preprint arXiv:2304.13734*, 2023.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[4] Yuanpu Cao, Tianrong Zhang, Bochuan Cao, Ziyi Yin, Lu Lin, Fenglong Ma, and Jinghui Chen. Personalized steering of large language models: Versatile steering vectors through bi-directional preference optimization. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 49519–49551. Curran Associates, Inc., 2024. doi: 10.52202/079017-1567. URL `https://proceedings.neurips.cc/paper_files/paper/2024/file/58cbe393b4254da8966780a40d023c0b-Paper-Conference.pdf`.

[5] Ekaterina Fadeeva, Aleksandr Rubashevskii, Artem Shelmanov, Sergey Petrakov, Haonan Li, Hamdy Mubarak, Evgenii Tsymbalov, Gleb Kuzmin, Alexander Panchenko, Timothy Baldwin, et al. Fact-checking the output of large language models via token-level uncertainty quantification. *arXiv preprint arXiv:2403.04696*, 2024.

[6] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[7] John Hewitt and Christopher D. Manning. A structural probe for finding syntax in word representations. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the*

Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4129–4138, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1419. URL `https://aclanthology.org/N19-1419/`.

[8] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of educational psychology*, 24(6):417, 1933.

[9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[10] Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis. *arXiv preprint arXiv:2405.07987*, 2024.

[11] Adam Tauman Kalai, Ofir Nachum, Santosh S Vempala, and Edwin Zhang. Why language models hallucinate. *arXiv preprint arXiv:2509.04664*, 2025.

[12] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[13] Nora Kassner, Oyvind Tafjord, Hinrich Schütze, and Peter Clark. BeliefBank: Adding memory to a pre-trained language model for a systematic notion of belief. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8849–8861, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.697. URL `https://aclanthology.org/2021.emnlp-main.697/`.

[14] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. Halueval: A large-scale hallucination evaluation benchmark for large language models. *arXiv preprint arXiv:2305.11747*, 2023.

[15] Potsawee Manakul, Adian Liusie, and Mark Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 9004–9017, 2023.

[16] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. *arXiv preprint arXiv:2310.06824*, 2023.

[17] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*, 2023.

[18] Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Askell, Samuel R Bowman, Newton Cheng, Esin Durmus, Zac Hatfield-Dodds, Scott R Johnston, et al. Towards understanding sycophancy in language models. *arXiv preprint arXiv:2310.13548*, 2023.

[19] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[20] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. ISSN 0925-2312. doi: https://doi.org/10.1016/j.neucom.2023.127063. URL https://www.sciencedirect.com/science/article/pii/S0925231223011864.

[21] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[22] Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL https://qwenlm.github.io/blog/qwen2.5/.

[23] TII Team. Falcon 3 family of open foundation models, December 2024.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Appendices